

- In the above code is coordinating two independent tasks GreenTaskA and TaskB that are talking to each other using a Semaphore.
- GreenTaskA has a higher priority than TaskB.
- When GreenTaskA calls vTaskDelay(), it blocks the task for 100ms i.e tasks goes into waiting state.
- While GreenTaskA is asleep, the CPU takes the next higher priority task in ready state which is TaskB.
- TaskB then tries to take semaphore. If the semaphore is not there then taskB also waits, and goes in the blocking state after some time as it is timebound.
- Once TaskB gets the CPU, it tries to take the semaphore. If the semaphore isn't ready, TaskB *also* goes to sleep after its time count ends.
- Because both tasks spend most of their time waiting for time to pass or waiting for a semaphore, so they share the CPU perfectly.
- Once the timeout hits, the processor executes the else part and says failed to get semaphore
- when the TaskB finally acquires the semaphore it is the red led on
- As the tasks are in an infinite loop it immediately loops back to the top of the function and keeps executing.